

OSSP

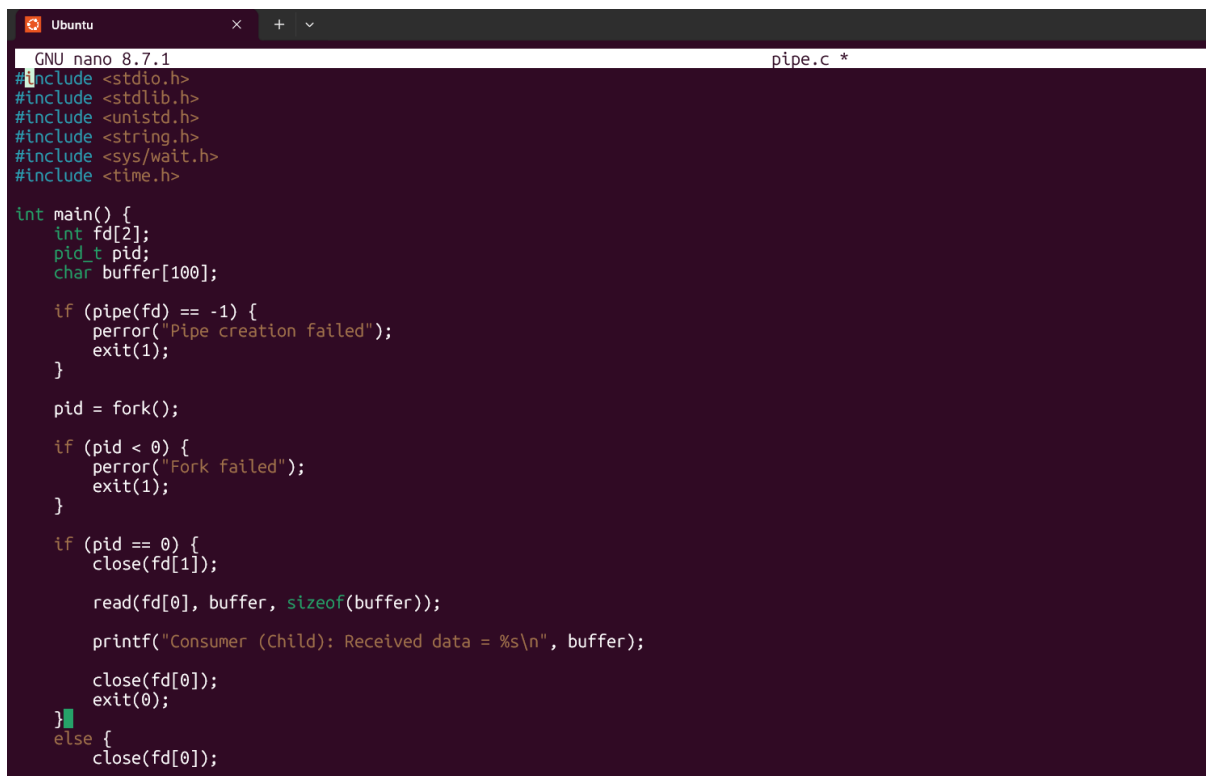
Practical Week -5

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.



```
GNU nano 8.7.1 pipe.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    int fd[2];
    pid_t pid;
    char buffer[100];

    if (pipe(fd) == -1) {
        perror("Pipe creation failed");
        exit(1);
    }

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    if (pid == 0) {
        close(fd[1]);

        read(fd[0], buffer, sizeof(buffer));

        printf("Consumer (Child): Received data = %s\n", buffer);

        close(fd[0]);
        exit(0);
    }
    else {
        close(fd[0]);
```

```
    else {
        close(fd[0]);

        strcpy(buffer, "Hello from Producer!");

        printf("Producer (Parent): Sending data = %s\n", buffer);

        int data_size = strlen(buffer) + 1;

        clock_t start = clock();

        write(fd[1], buffer, data_size);

        clock_t end = clock();

        double communication_time =
            (double)(end - start) / CLOCKS_PER_SEC;

        printf("Data size: %d bytes\n", data_size);
        printf("Communication time: %f seconds\n",
            communication_time);

        if (communication_time > 0) {
            double throughput =
                data_size / communication_time;

            printf("Communication throughput: %f bytes/second\n",
                throughput);
        }

        close(fd[1]);

        wait(NULL);

        printf("Producer: Consumer process completed.\n");
    }
}
```

```
Ubuntu × + ▾
GNU nano 8.7.1 pipe.c *

    printf("Producer (Parent): Sending data = %s\n", buffer);

    int data_size = strlen(buffer) + 1;

    clock_t start = clock();

    write(fd[1], buffer, data_size);

    clock_t end = clock();

    double communication_time =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("Data size: %d bytes\n", data_size);
    printf("Communication time: %f seconds\n",
        communication_time);

    if (communication_time > 0) {
        double throughput =
            data_size / communication_time;

        printf("Communication throughput: %f bytes/second\n",
            throughput);
    }

    close(fd[1]);

    wait(NULL);

    printf("Producer: Consumer process completed.\n");
}

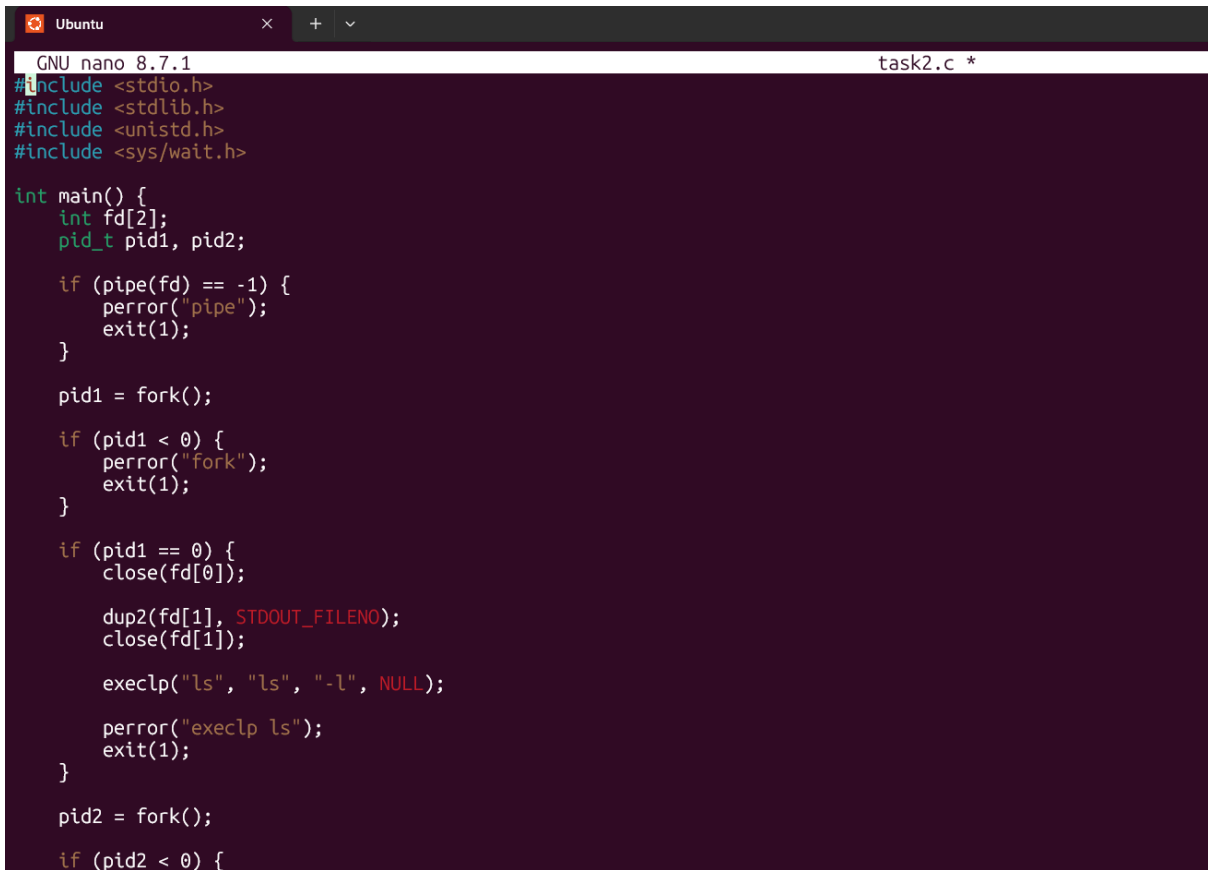
return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Lo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go
```

```
nocturnal@nocturnal:~$ nano pipe.c
nocturnal@nocturnal:~$ gcc pipe.c -o pipe
nocturnal@nocturnal:~$ ./pipe
Producer (Parent): Sending data = Hello from Producer!
Data size: 21 bytes
Communication time: 0.000003 seconds
Communication throughput: 7000000.000000 bytes/second
Consumer (Child): Received data = Hello from Producer!
Producer: Consumer process completed.
nocturnal@nocturnal:~$
```

Task-2

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.



```
GNU nano 8.7.1 task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1) {
        perror("pipe");
        exit(1);
    }

    pid1 = fork();

    if (pid1 < 0) {
        perror("fork");
        exit(1);
    }

    if (pid1 == 0) {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid2 = fork();

    if (pid2 < 0) {
```

GNU nano 8.7.1

```
    close(fd[1]);

    execlp("ls", "ls", "-l", NULL);

    perror("execlp ls");
    exit(1);
}

pid2 = fork();

if (pid2 < 0) {
    perror("fork");
    exit(1);
}

if (pid2 == 0) {
    close(fd[1]);

    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);

    execlp("grep", "grep", ".c", NULL);

    perror("execlp grep");
    exit(1);
}

close(fd[0]);
close(fd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

return 0;
```

```
}
```

```
nocturnal@nocturnal:~$ nano task2.c
nocturnal@nocturnal:~$ gcc task2.c -o task2
nocturnal@nocturnal:~$ ./task2
drwxrwxrwx 6 nocturnal nocturnal 4096 Aug 14 14:35 OSSP_2520090085
-rwxr-xr-x 1 nocturnal nocturnal 16464 Sep 12 03:44 pipe
-rw-r--r-- 1 nocturnal nocturnal 1457 Sep 12 03:44 pipe.c
-rwxr-xr-x 1 nocturnal nocturnal 16288 Sep 12 03:21 task1
-rw-r--r-- 1 nocturnal nocturnal 903 Sep 12 03:21 task1.c
-rwxr-xr-x 1 nocturnal nocturnal 16312 Sep 12 03:24 task1_waitpid
-rw-r--r-- 1 nocturnal nocturnal 903 Sep 12 03:24 task1_waitpid.c
-rwxr-xr-x 1 nocturnal nocturnal 16296 Sep 12 03:46 task2
-rw-r--r-- 1 nocturnal nocturnal 892 Sep 12 03:45 task2.c
-rwxr-xr-x 1 nocturnal nocturnal 16208 Sep 12 03:36 zombie
-rw-r--r-- 1 nocturnal nocturnal 684 Sep 12 03:35 zombie.c
nocturnal@nocturnal:~$
```